Routing an Ambulance through a Grid-Structured City: Integer Programming, Sensitivity Analysis, and A* Search

A Two-Leg Shortest Path Project on Manhattan-Plan Road Networks

Lampros Prinzios
Department of . . . , University of . . . , . . .@. . .
Final project for the course "Linear and Combinatorial Optimization"

Abstract

We study a two-leg ambulance routing problem on city road networks with a rectangular (Manhattan) street plan: an ambulance must first reach a patient and then transport the patient to a hospital, so that each leg is a single-pair shortest path problem on a directed, edge-weighted grid network. Each leg is modeled as a minimum-cost-flow-type integer program (IP) with binary arc variables and flow-conservation constraints, which is solved with the GLPK solver through the pymprog modeling layer. We establish several structural properties of the model: the sink-side conservation constraint and all explicit degree/subtour constraints are redundant; the constraint matrix is the node–arc incidence matrix of a digraph and is therefore totally unimodular, so the linear programming relaxation is exact; and the two-leg problem decomposes, by Bellman's optimality principle, into two independent IPs that share the same constraint matrix and cost vector and differ only in the right-hand side. Both minimization criteria of the underlying tool are covered—route length and travel time—the latter induced by additive, integer per-segment time penalties (traffic, road works, gravel roads, traffic lights). As an algorithmic alternative we implement A* search with a Manhattan-distance heuristic and prove its admissibility and consistency for both criteria. Finally, we develop an integer sensitivity analysis that determines, for every edge, the maximal cost perturbation that the current shortest path tolerates before rerouting; a binary search over a monotone objective-value predicate locates this threshold with $\lceil \log_2 N \rceil$ IP re-solves instead of the N re-solves of a linear scan, and a sufficient condition for uniqueness of the optimal path is derived. The optima obtained by the IP solver, by Dijkstra's algorithm, and by A* agree on all five benchmark instances, with grids of up to 100×100 cells. All components are integrated in an interactive graphical tool for constructing, solving, and analyzing such networks.

1 Introduction

Emergency medical response is one of the canonical applications of network optimization: when an ambulance is dispatched to a patient who must subsequently be transported to a hospital, every second of travel time matters. In cities with a rectangular (grid or "Manhattan") street plan, the road network is naturally represented as a graph whose vertices are junctions and whose edges are road segments, with costs modeling either geometric length or expected travel time. The operational question is then a *two-leg* routing question: a shortest path from the ambulance to the patient, followed by a shortest path from the patient to the hospital.

This paper documents, formalizes, and evaluates a complete project that addresses exactly this scenario. A graphical tool allows the user to draw road networks on a configurable grid, to attach directedness and time penalties to road segments, to place the three special vertices (ambulance, patient, hospital), and to solve both legs of the routing problem either as integer programs or with heuristic search. The project's contributions, which we develop and assess in this paper, are the following.

- **A modeling framework** (Section 3) that turns an interactive grid drawing into a formal optimization instance: undirected segments are split into antiparallel arcs, per-unit time penalties of several categories are aggregated into integer edge weights, and both length- and time-based criteria are supported with an exact conversion between relative cost units and physical seconds or meters.
- **An integer programming formulation** of each leg (Section 4) together with a structural analysis: redundancy of the sink constraint (Proposition 1), redundancy of degree and subtour constraints under positive costs (Proposition 2), exactness of the LP relaxation via total unimodularity (Proposition 3), and a decomposition of the two-leg problem into two IPs that differ only in their right-hand sides (Proposi-

tion 4, Remark 1).

- **An A* implementation** for single-pair queries (Section 5), with a Manhattan-distance heuristic whose admissibility and consistency we prove for both criteria (Proposition 5).
- **An integer sensitivity analysis** (Section 6) that computes, for each edge, the threshold $\Delta c_e^{\max}$ on its cost perturbation before the optimal route changes, characterizes the value function as a concave piecewise-linear function of the perturbation, accelerates the search for the integer threshold from N to $\lceil \log_2 N \rceil$ IP re-solves via binary search on a monotone objective-value predicate, and yields a certificate of uniqueness of the optimal path (Remark 2).
- **An experimental evaluation** (Section 7) on five instances with grids of up to 100×100 cells, cross-validating the optima returned by three independent solvers (GLPK through pymprog, Dijkstra's algorithm, and A*), and a full sensitivity study on a worked example, including the comparison of the linear and binary search variants (9 versus 4 re-solves).

All results reported in this paper were verified against the outputs of the project's own solvers. The remainder of the paper is organized as follows. Section 2 reviews related work. Sections 3–6 develop the methodology. Section 7 reports the experiments, Section 8 discusses scope, novelty, and limitations, and Section 9 concludes.

2 Related work

Shortest path algorithms. The single-source and single-pair shortest path problems are among the oldest and most thoroughly studied problems of network optimization; Schrijver [1] traces their history back to the nineteenth century. Bellman [2] characterized the problem through dynamic programming, Ford [3] described the label-correcting method, and Dijkstra [4] gave the classical greedy label-setting algorithm. Dreyfus [5] surveyed the early algorithm families. Our A* implementation follows Hart, Nilsson, and Raphael [6], whose admissible heuristics guarantee optimality; Dechter and Pearl [7] analyzed the optimality and efficiency of A* and its variants in depth. On road networks specifically, Goldberg and Harrelson [8] popularized the combination of A* with landmark-based lower bounds; our heuristic is the structurally analogous, metric lower bound supplied by the grid geometry itself (the Manhattan metric [9]).

Shortest paths as linear and integer programs. Dantzig [10] and Ford and Fulkerson [11] observed early that shortest path problems are special minimum-cost flow problems and hence linear programs. The systematic polyhedral theory—in particular the total unimodularity of node–arc incidence matrices, which makes the LP relaxation exact—is classical and is treated in the monographs of Ford and Fulkerson [11], Nemhauser and Wolsey [12], Ahuja, Magnanti, and Orlin [13], and Schrijver [14]. Textbooks that connect the network-flow view with the communication-network applications we draw upon include Oki [15], and Taccari [16] compares practical integer programming formulations of elementary shortest path problems, including the formulation we use.

Parametric and sensitivity aspects. Sensitivity analysis is a classical topic of linear programming [17]. For shortest paths specifically, parametric versions in which one edge cost varies linearly were studied by Karp and Orlin [18] and accelerated by Young, Tarjan, and Orlin [19]; Carstensen [20] showed that parametric linear and combinatorial programs can have superpolynomially many breakpoints. Our analysis (Section 6) is deliberately more elementary: it computes a single integer threshold per edge by re-solving, which keeps the method solver-agnostic (any black-box IP or LP solver can be used, and no dual information is required); we position this choice in Section 8.

Emergency medical logistics. Our scenario is the single-vehicle core of the much richer field of ambulance location and dispatch. Larson's hypercube model [21] initiated the queueing-theoretic analysis of urban emergency services; Brotcorne, Laporte, and Semet [22] survey ambulance location and relocation models, and Gendreau, Laporte, and Semet [23] treat real-time relocation heuristically. Time-dependent travel times, which motivate our time penalties, are treated exactly by Orda and Rom [24]. We deliberately keep the network static and deterministic, and focus on the optimization-theoretic core of the routing question.

3 Scenario and network model

3.1 City grids and road networks

The tool's canvas is an $R \times C$ grid of square cells, the unit size s of which can be chosen from $\{1, 2, 5, 10, 20, 50, 100\}$ meters; a 10×10 grid with $s = 1\,\text{m}$ therefore models a city block of 10×10 square meters. The user places *vertices* (crossroads) on cells and connects them by *edges* (roads): an edge between two vertices is drawn as a horizontal or vertical sequence of adjacent cells, and each edge carries

- a *directionality*—one-way in either of the two axis directions (rendered $\rightarrow, \leftarrow, \uparrow, \downarrow$) or two-way

($\leftrightarrow$); every undirected edge is split into the two antiparallel arcs of the directed model, and

- optional *time penalties*: each cell of the edge may carry one add-on category (*traffic*, *road works*, *gravel road*, *traffic lights*), each equipped with an integer multiplier $\omega \geq 2$ (the plain cells carry the default multiplier $\omega = 1$).

Three vertices are marked as the *ambulance* A, the *patient* P, and the *hospital* H. The graph is checked for strong connectivity with depth-first search before solving; if P is not reachable from A, or H is not reachable from P, the instance is rejected as infeasible.

Formally, the road network defines a directed graph $G = (V, \mathcal{A})$ with $|V| = n$ vertices and m arcs: every edge $\{u, v\}$ of the drawing contributes the arc (u, v), the arc (v, u), or both, according to its directionality. We write $\delta^+(v)$ and $\delta^-(v)$ for the arcs leaving and entering v.

3.2 Two cost criteria

Two minimization criteria can be selected, and both are integral in nature.

Length. The length cost of an arc $a = (u, v)$ is the Manhattan distance of its endpoints,

$$w_a^{\text{len}} \;=\; d_{\text{Man}}(u, v) \;=\; |u_1 - v_1| + |u_2 - v_2|,$$

i.e., the number of grid units the corresponding road spans, since every edge is an axis-parallel chain between its endpoints. Absolute lengths are obtained by multiplying with the unit size s (meters).

Time. The time cost of an arc is the sum of the multipliers of all cells of its road plus one unit for the road itself:

$$w_a^{\text{time}} \;=\; 1 + \sum_{\text{cell } \gamma \in a} \omega_\gamma, \qquad \omega_\gamma \in \{1, 2, \dots\}.$$

Thus a plain road spanning k cells costs $k + 1$, and each penalty cell adds $\omega_\gamma - 1$ extra units. The tool reports absolute travel times by converting relative units through the *normal time cost per unit*

$$\text{n.t.c.} \;=\; \frac{3.6\, s}{v} \quad \text{seconds},$$

where $v \in \{10, 20, \dots, 60\}$ km/h is the ambulance's cruising speed: a route of relative time cost Z takes $Z \cdot$ n.t.c. seconds. For example, on a 1-meter grid at $v = 10$ km/h each unit costs 0.36 s.

Both criteria are aggregated additively along a path, and in both cases the cost of an arc dominates the Manhattan distance of its endpoints; this fact powers the admissibility proof of Section 5. We write $w_a > 0$ for the arc cost under the selected criterion, $C(P) = \sum_{a \in P} w_a$ for the cost of a path P, and $Z = \min_{P: s \to t} C(P)$ for the optimal value of a single-pair shortest path problem with source s and target t.

4 Integer programming for the two legs

4.1 Formulation

For a fixed leg (first $A \to P$, then $P \to H$) with source s and target t, we introduce one binary variable per arc, $x_a \in \{0, 1\}$, indicating whether arc a is used, and impose flow conservation of one unit of flow:

$$\begin{aligned} \min \quad & \sum_{a \in \mathcal{A}} w_a\, x_a \\ \text{s.t.} \quad & \sum_{a \in \delta^+(v)} x_a \;-\; \sum_{a \in \delta^-(v)} x_a \;=\; b_v, \quad v \in V, \\ & x_a \in \{0, 1\}, \qquad\qquad\qquad\qquad a \in \mathcal{A}, \end{aligned} \tag{1}$$

with right-hand side

$$b_v \;=\; \begin{cases} 1 & v = s, \\ -1 & v = t, \\ 0 & \text{otherwise.} \end{cases}$$

In matrix form, $Ax = b$ where $A \in \{-1, 0, 1\}^{n \times m}$ is the (unsigned incidence-based) coefficient matrix with, in column $a = (u, v)$, a $+1$ in row u and a -1 in row v. The models are handed to the GLPK solver [25] through the pymprog modeling layer, which writes both the model and the solution to disk for inspection (the solver uses its long-step dual simplex on the LP relaxation).

4.2 Structural properties

Proposition 1 (The sink constraint is redundant)**.** *In system* (1)*, the conservation constraint of the target vertex t is implied by the constraints of all other vertices.*

Proof. Each column of A contains exactly one $+1$ and one -1, hence the rows of A sum to the zero vector, and consequently also b sums to zero. The row of t equals the negative sum of the rows of $V \setminus \{t\}$, and the same identity holds between the two corresponding sides of $Ax = b$; any x satisfying all constraints except that of t therefore satisfies it as well. □

Proposition 2 (Degree and subtour constraints are redundant)**.** *Every feasible binary solution of* (1) *is the incidence vector of a vertex-disjoint union of one $s-t$ path and a (possibly empty) set of directed cycles. Consequently, under the positive costs $w_a > 0$, every* optimal *solution is a simple $s-t$ path, and no explicit degree constraints ("at most one outgoing arc per vertex") or subtour elimination constraints are needed.*

Proof. Let $x \in \{0, 1\}^{\mathcal{A}}$ satisfy $Ax = b$. Consider the subgraph induced by $\{a : x_a = 1\}$: every vertex v

has out-degree minus in-degree b_v, so following arcs forward from s (which has surplus $+1$) traces an $s-t$ walk; since x is binary the walk is a path plus possibly cycles through other vertices. Removing the arcs of any cycle keeps $Ax = b$ satisfied and strictly decreases the objective because all costs are positive, so no cycle appears in an optimal solution. The same argument excludes repeated vertices. □

Proposition 3 (The LP relaxation is exact)**.** *A is the node–arc incidence matrix of the digraph G. It is totally unimodular, and therefore the linear relaxation of (1) (with $0 \le x_a \le 1$) has an integral optimal solution of the same value.*

Proof. Total unimodularity of node–arc incidence matrices of directed graphs is a classical result [11, 13, 14]: every square submatrix has determinant in $\{-1, 0, 1\}$, because each column has at most one $+1$ and at most one -1. Since b is integral, all vertices of the polyhedron $\{x : Ax = b,\ 0 \le x \le 1\}$ are integral, so the LP optimum is attained at a binary point. Together with Proposition 2, the LP relaxation solves the integer program exactly, with no branching required. □

Proposition 4 (Two-leg decomposition)**.** *The minimum-cost route from A to H that visits P equals the sum of the minima of the two independent single-pair problems $A \to P$ and $P \to H$.*

Proof. Any $A \to H$ route visiting P splits at P into an $A \to P$ part and a $P \to H$ part with additive cost, so the route minimum is at least $Z_{A\to P} + Z_{P\to H}$; concatenating optimal paths for the two parts attains the bound (Bellman's optimality principle [2]). No other interaction exists between the legs: congesting the first leg does not affect the second. □

Remark 1 (Only the right-hand side changes)**.** *The matrix A and the cost vector $c = (w_a)_a$ depend only on the road network and the selected criterion; the two legs differ exclusively in b: $(b_{A\to P})_v$ has $+1$ at A and -1 at P, while $(b_{P\to H})_v$ has $+1$ at P and -1 at H. In the tool, both IPs are generated from the same data structures and solved in succession.*

4.3 A worked toy instance

Figure 1 shows the network used throughout the project as its running example: 5 vertices, 8 roads, and 10 arcs after splitting the two-way roads. With the time criterion and unit multipliers $\omega_{\text{traffic}} = \omega_{\text{road works}} = 2$, $\omega_{\text{gravel}} = 3$, the cost vector is $c = (4, 5, 2, 6, 5, 2, 7, 2, 6, 3)^\top$ for the variables $(x_{12}, x_{13}, x_{14}, x_{25}, x_{31}, x_{34}, x_{35}, x_{42}, x_{52}, x_{54})$, and the two right-hand sides are $b_{A\to P} = (1, 0, 0, 0, -1)^\top$ and $b_{P\to H} = (0, -1, 0, 0, 1)^\top$. Solving (1) with GLPK returns $Z_{A\to P} = 10$ (attained *twice*, by $x_{12} = x_{25} = 1$ and by $x_{14} = x_{42} = x_{25} = 1$) and $Z_{P\to H} = 5$ (attained by $x_{54} = x_{42} = 1$). At the default parameters (1 m cells, 10 km/h) these are 3.6 s and 1.8 s of absolute travel time.

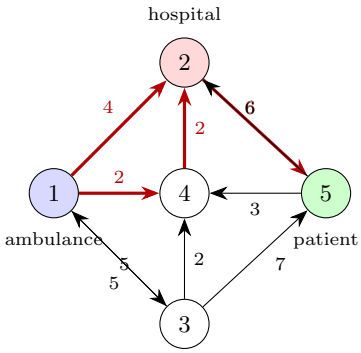


Figure 1: The 5-vertex, 10-arc toy network of the tool (Section 4.3), with arc costs. Bold arcs form the two optimal ambulance → patient paths $1 \to 2 \to 5$ and $1 \to 4 \to 2 \to 5$, both of cost 10; the optimal patient → hospital path $5 \to 4 \to 2$ (dashed bold in the tool's rendering) costs $3 + 2 = 5$.

5 A* search with a Manhattan heuristic

For interactive single-pair queries the tool also implements A* search [6]: nodes are expanded in increasing order of $f(n) = g(n) + h(n)$, where $g(n)$ is the cost incurred from the source to n and $h(n)$ estimates the remaining cost to the target. Dijkstra's algorithm [4] is the special case $h \equiv 0$; A* is preferable here because exactly one target matters, whereas Dijkstra necessarily computes distances to all vertices. Both algorithms are complete on finite graphs with non-negative costs.

The heuristic is the Manhattan distance to the target, $h(n) = d_{\text{Man}}(n, t)$: on a rectangular street plan, the length of *any* route between two vertices is at least their ℓ_1-distance [9], so the estimate is structurally reliable rather than merely plausible.

Proposition 5 (Admissibility and consistency of the Manhattan heuristic)**.** *For both criteria of Section 3, $h(n) = d_{\text{Man}}(n, t)$ is admissible, i.e. $h(n) \le \text{dist}(n, t)$ for all n, and consistent, i.e. $h(u) \le w(u, v) + h(v)$ for every arc (u, v).*

Proof. Every arc $a = (u, v)$ satisfies $w_a \ge d_{\text{Man}}(u, v)$: under the length criterion equality holds by construction; under the time criterion $w_a = 1 + \sum_{\gamma \in a} \omega_\gamma \ge k_a + 1 > k_a \ge d_{\text{Man}}(u, v)$, where k_a is the number of cells of a's road, which any connected cell chain between u and v must at least cover. Summing along any $n \to t$ path and applying the triangle inequality of the ℓ_1 metric gives $\text{dist}(n, t) \ge d_{\text{Man}}(n, t)$: admissibility. Consistency follows from the same triangle inequality, $d_{\text{Man}}(u, t) \le d_{\text{Man}}(u, v) + d_{\text{Man}}(v, t) \le w(u, v) + h(v)$. □

Consequently, A* with this heuristic is optimal (it returns minimum-cost paths) on our networks under both criteria [6, 7]. The implementation maintains open lists, parent pointers, and g-scores in dictionaries over grid coordinates and resolves each road's cost

by a lookup keyed by the vertex pair; on all instances of Section 7 its outputs agreed with the IP optima.

6 Sensitivity analysis

The travel-time weights of a road network are volatile: traffic builds up and dissipates, road works appear and end. It is therefore natural to ask, for each edge e of the current solution, *how much its cost may change before the optimal route changes*. The tool answers this with an integer sensitivity analysis: the perturbations applied are the integer increments and decrements of an edge's relative cost, matching the granularity at which the add-on multipliers themselves can be edited. Throughout, let p_{old} be the current optimal $s \to t$ path of value Z.

6.1 Perturbing an edge on the optimal path

Proposition 6 (Threshold for cost increases)**.** *Let $e \in p_{old}$ and let $M_e = \min\{C(P) : P$ an $s \to t$ path with $e \notin P\}$ (if no such P exists, set $M_e = \infty$). For an integer increase $\delta \geq 0$ of c_e:*

(i) the perturbed optimum is $Z(\delta) = \min\{Z + \delta,\ M_e\}$, a concave piecewise-linear function of δ with a single breakpoint (Figure 2a);

(ii) p_{old} remains optimal if and only if $0 \leq \delta \leq \Delta c_e^{\max}$, where

$$\Delta c_e^{\max} = M_e - Z \geq 0, \tag{2}$$

with ties possible at $\delta = \Delta c_e^{\max}$; for $\delta = \Delta c_e^{\max} + 1$ the strictly better alternative avoiding e takes over.

Proof. Paths avoiding e are unaffected by δ, so their best value stays M_e; every path through e shifts up by exactly δ, and the best of them is p_{old} itself with $C(p_{\text{old}}) + \delta = Z + \delta$. Hence $Z(\delta) = \min\{Z + \delta, M_e\}$, and $Z + \delta \leq M_e$ iff $\delta \leq M_e - Z$. □

If e is a bridge (every $s \to t$ path uses e), then $M_e = \infty$ and the tolerance is unbounded: the network can absorb any increase of c_e without rerouting.

6.2 Perturbing an edge off the optimal path

Proposition 7 (Threshold for cost decreases)**.** *Let $e \notin p_{old}$ and let $M'_e = \min\{C(P) : P$ an $s \to t$ path with $e \in P\}$. For an integer decrease of c_e by $\delta \geq 0$ (bounded below by the minimum achievable cost ℓ_e of e, i.e. $c_e - \delta \geq \ell_e$): the perturbed optimum is $Z(\delta) = \min\{Z,\ M'_e - \delta\}$, again concave piecewise-linear (Figure 2b), and p_{old} remains optimal iff $0 \leq \delta \leq \Delta c_e^{\max-}$ with $\Delta c_e^{\max-} = Z - M'_e \geq 0$. At $\delta = \Delta c_e^{\max-}$ a path through e ties with p_{old}; beyond it, each further unit of reduction decreases the optimum by exactly one unit along the new path (which traverses e once). For the time criterion, $\ell_e = k_e + 1$ where k_e is the number of cells of e's road (all multipliers at their default $\omega = 1$); in particular the geometric floor $d_{\text{Man}}(u_e, v_e) \leq \ell_e$ from Section 5 bounds any reduction.*

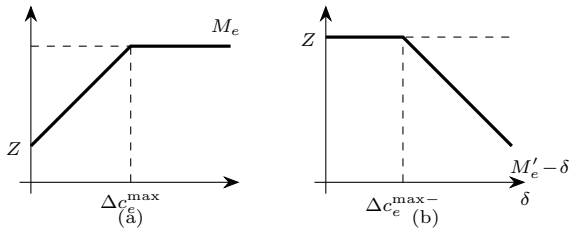


Figure 2: The value function of the perturbed problem. (a) Increasing the cost of an edge e on the optimal path: $Z(\delta) = \min\{Z + \delta,\ M_e\}$ with breakpoint $\Delta c_e^{\max} = M_e - Z$. (b) Decreasing the cost of an edge off the optimal path: $Z(\delta) = \min\{Z,\ M'_e - \delta\}$ with breakpoint $\Delta c_e^{\max-} = Z - M'_e$, until the floor ℓ_e of e is reached.

Proof. Symmetric to Proposition 6: the best path through e now has value $M'_e - \delta$ while all others, including p_{old}, keep their costs; the minimum of the two families gives $Z(\delta)$ and the crossing point $Z - M'_e$. Once $M'_e - \delta < Z$ the optimum uses e, and decreasing δ further lowers the value one unit per unit since e is traversed once. The floor ℓ_e holds because multipliers are integers ≥ 1. □

6.3 Locating the integer threshold by binary search

The thresholds of Propositions 6 and 7 can be found by increasing (respectively decreasing) the edge cost in steps of one unit and re-solving model (1) after each step until the returned optimum signals a reroute. This linear scan requires up to N IP re-solves, where N is an a priori bound on the relevant perturbation range (e.g. the distance of c_e from its extreme achievable value). Both value functions of Figure 2 are monotone after their single breakpoint, so the "still optimal" predicate is *monotone* in δ, and binary search applies.

The predicate in line 5 compares *objective values* only—$Z(\delta) = Z + \delta$ in the increase case, $Z(\delta) = Z$ in the decrease case—which by Propositions 6 and 7 holds exactly up to the threshold and fails afterwards; this makes the test robust to ties (at $\delta = \Delta c_e^{\max}$ the solver may return either of the tied paths, but the optimal *value* is unambiguous). The loop performs at most $\lceil \log_2 N \rceil$ IP re-solves.

Remark 2 (A certificate of uniqueness)**.** *If, for* every *edge e of p_{old}, the cost of e can be driven all the way to its maximum achievable value without the optimal route changing (i.e. the tolerance of e covers its entire admissible range), then p_{old} is the* unique *optimal $s \to t$ path. Indeed, any simple alternative path*

Algorithm 1 Integer sensitivity threshold via binary search.

Input: solved instance (A, b, c) with optimal value Z; edge e; bound N; direction $d \in \{+1, -1\}$.
Output: $\Delta c_e^{\max}$ (increase case $d = +1$) or $\Delta c_e^{\max-}$ (decrease case $d = -1$).

1. $lo \leftarrow 0$; $hi \leftarrow N$.
2. **while** $lo < hi$ **do**
3. $\quad mid \leftarrow \lceil (lo + hi + 1)/2 \rceil$.
4. $\quad$ solve (1) with c_e replaced by $c_e + d \cdot mid$; let $Z(mid)$ be its optimal value.
5. $\quad$ **if** $Z(mid) = Z + d \cdot mid$ **then** $lo \leftarrow mid$ **else** $hi \leftarrow mid - 1$.
6. **return** $\Delta \leftarrow lo$.

Table 1: The five benchmark instances. $|E|$ counts roads, $|\mathcal{A}|$ the arcs after splitting two-way roads; Z are the relative optimal costs of the two legs as computed by the IP (GLPK), cross-validated with Dijkstra's algorithm and with A* (agreement on all values and paths). The optima of E3 and E5 were produced with the same model data by an independent shortest-path implementation, which coincides with the IP optimum by Proposition 3.

Inst.	Grid	$\lvert V\rvert$	$\lvert E\rvert$	$\lvert\mathcal{A}\rvert$	Crit.	$Z_{A\to P}$	$Z_{P\to H}$
E1	10×10	5	8	10	time	10	5
E2	30×30	14	16	26	length	24	52
E3	20×20	9	12	24	time	42	32
E4	50×50	28	32	64	time	205	258
E5	100×100	23	27	54	length	193	271

either strictly contains p_{old}'s arc set (and is costlier, by positivity of the costs) or omits some edge $e \in p_{old}$, in which case its cost is at least $Z + \Delta c_e^{\max} > Z$. Running the sensitivity analysis over all edges of the incumbent path therefore doubles as a uniqueness test.

7 Experimental evaluation

7.1 Tool and instances

All experiments were run with the project's tool: a Tkinter-based graphical application in which the user draws the network, selects the criterion, checks connectivity, solves the two legs (IP or A*), and applies the sensitivity analysis per edge; instances are stored as plain-text files listing vertices, edge cells, penalty categories with their multipliers, and directionalities. Table 1 summarizes the five benchmark instances: grid sizes from 10×10 to 100×100 cells, 5 to 28 vertices, 8 to 32 roads (10 to 64 arcs after splitting two-way roads), and both criteria.

On E1 (Figure 1) the ambulance → patient leg has the two tied optima of cost 10 discussed in Section 4.3. On E2, the geometrically shorter patient → hospital connection of length 30 is infeasible because one of its roads is one-way in the opposite direction, and the optimal feasible path costs 52—an illustration of why directionality must be modeled explicitly. On E4, the largest time-criterion instance, GLPK certifies optima 205 and 258 for the two legs, matching the A* and Dijkstra solutions.

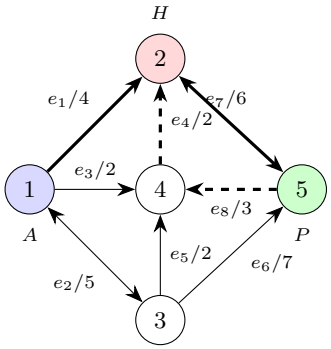


Figure 3: The sensitivity experiment network (same vertex layout as Figure 1); edge labels give the edge id and its time cost. Bold solid arcs form the optimal ambulance → patient path $\{e_1, e_7\}$ (cost 10); bold dashed arcs form the optimal patient → hospital path $\{e_8, e_4\}$ (cost 5).

Table 2: Sensitivity thresholds on the network of Figure 3; "inc."/"dec." marks an increase/decrease analysis on an edge of the incumbent optimal path (the $A \to P$ incumbent is $\{e_1, e_7\}$, the $P \to H$ incumbent $\{e_8, e_4\}$, and in the decrease scenario the incumbent is $\{e_7\}$). "Reroute at $\Delta c^{\max} + 1$" names the alternative that becomes strictly better one unit past the threshold (its cost under the unperturbed costs in parentheses). The last row is the decrease case on the modified instance with a gravel penalty on e_8 (making e_8 cost 5 and $\{e_7\}$ optimal at cost 6).

Edge	Leg	Type	$\Delta c^{\max}$	Reroute at $\Delta c^{\max} + 1$
e_1	$A \to P$	inc.	0	$\{e_3, e_4, e_7\}$ (10)
e_7	$A \to P$	inc.	2	$\{e_2, e_6\}$ (12)
e_4	$P \to H$	inc.	1	$\{e_7\}$ (6)
e_8	$P \to H$	inc.	1	$\{e_7\}$ (6)
e_8	$P \to H$	dec.	1	$\{e_8, e_4\}$ $(7 \to 5)$

7.2 The sensitivity experiment

Figure 3 shows the network of the sensitivity study: the same 5-vertex, 8-road network under the time criterion, with two-way roads e_2 and e_7. There are four $A \to P$ paths—$\{e_1, e_7\}$ and $\{e_3, e_4, e_7\}$, both of cost 10, then $\{e_2, e_6\}$ (12) and $\{e_2, e_5, e_4, e_7\}$ (15)—and two $P \to H$ paths: $\{e_7\}$ (6) and $\{e_8, e_4\}$ (5). The incumbent optima are $\{e_1, e_7\}$ and $\{e_8, e_4\}$. Table 2 reports the thresholds produced by the analysis.

All values confirm the closed forms of Propositions 6 and 7: for instance, increasing c_{e_7} leaves both 10-cost paths through e_7 intact while the best e_7-

Table 3: Linear scan versus binary search for the integer threshold on the edge of Figure 3 analyzed in the project (perturbation bound $N = 12$).

Search strategy	IP re-solves	A priori bound
Linear scan ($\delta = 0, 1, 2, \dots$)	9	$\leq N = 12$
Binary search (Algorithm 1)	4	$\lceil \log_2 12 \rceil = 4$

free path costs 12, hence $\Delta c_{e_7}^{\max} = 12 - 10 = 2$, and indeed at $\Delta c_{e_7} = 3$ the route flips to $\{e_2, e_6\}$. In the decrease scenario, placing a gravel penalty on e_8 (cost 5) makes $\{e_7\}$ optimal at cost 6; one unit of reduction on e_8 produces a tie ($6 = 6$), two units reroute to $\{e_8, e_4\}$ at cost 5—and two units is also the maximum possible reduction, since e_8 spans $k = 2$ cells whose multipliers cannot go below the default, i.e. its floor is $\ell_{e_8} = k + 1 = 3$.

Finally, Table 3 compares the two search modes of Section 6 on the same edge: the linear scan located the threshold at its 8-th probe and terminated at the 9-th (9 IP re-solves in total, with bound $N = 12$), whereas the binary search located it in 4 re-solves—exactly the $\lceil \log_2 12 \rceil = 4$ guaranteed by the analysis, with the termination check in its 5-th iteration. The gap widens with the perturbation bound, which matters when the analysis is run over all edges of large networks.

8 Discussion and limitations

What is classical and what is not. The ingredients of this project are individually classical: the minimum-cost-flow-type formulation goes back to Dantzig and Ford–Fulkerson [10, 11], the total-unimodularity argument is textbook material [13, 14], A* with a consistent heuristic is textbook material [6, 7], and sensitivity analysis of linear programs is classical [17]. The value of the project lies in the combination: a single coherent framework in which a drawing becomes a formal IP, both legs are solved and cross-validated, and a tie-robust, solver-agnostic integer sensitivity analysis with a binary-search acceleration is layered on top, together with a uniqueness certificate.

Re-solving versus ranging. Because the LP relaxation is exact (Proposition 3), classical LP objective ranging could in principle read the threshold (2) directly off the final simplex basis, without any re-solving; and parametric shortest path algorithms can trace the full value function for one varying edge in polynomial time [18, 19], with the caveat that the number of breakpoints can be large [20]. The re-solve-based method implemented here trades this theoretical elegance for two practical advantages: it works with any black-box solver (or even with the A* engine) that returns only optimal values, and it extends unchanged to integer perturbations of the combinatorial add-on structure, where ranging information is unavailable. A systematic empirical comparison with LP ranging is an obvious next step.

Limitations. The experiments are small ($\leq$ 28 vertices, $\leq$ 64 arcs) compared with real road networks, on which specialized techniques dominate [8]; the travel-time model is static and deterministic, while real travel times are time-dependent and stochastic [24]; and the sensitivity analysis varies one edge at a time, whereas realistic disruptions (weather, events) move many costs at once. The scenario itself is the single-vehicle core of a much larger operations problem, in which ambulance fleet location and dispatch interact [21, 22, 23]. These are directions for future work, not obstacles to the present tool's use case: interactive, educational exploration of the optimization models involved.

9 Conclusion

We presented a complete modeling-to-analysis pipeline for a two-leg ambulance routing scenario on grid-structured city networks: an integer programming formulation with proved redundancy, integrality, and decomposition properties; an optimally-admissible A* search component; and an integer sensitivity analysis whose thresholds are located with $\lceil \log_2 N \rceil$ re-solves and double as uniqueness certificates. Three independent solvers agreed on all benchmark instances, and the worked example confirmed every closed form. The project demonstrates how the classical theory of network flows, polyhedral integrality, and heuristic search fits together in one small, fully functional application.

References

[1] Alexander Schrijver. On the history of the shortest path problem. *Documenta Mathematica*, Extra Volume ISMP (Optimization Stories):155–167, 2012.

[2] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.

[3] Lester R. Ford, Jr. Network flow theory. Technical Report P-923, The RAND Corporation, Santa Monica, CA, 1956.

[4] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[5] Stuart E. Dreyfus. An appraisal of some shortest-path algorithms. *Operations Research*, 17(3):395–412, 1969.

[6] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[7] Rina Dechter and Judea Pearl. Generalized best-first search strategies and the optimality of A*. *Journal of the ACM*, 32(3):505–536, 1985.

[8] Andrew V. Goldberg and Chris Harrelson. Computing the shortest path: A* search meets graph theory. In *Proceedings of the 16th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 156–165, 2005.

[9] Eugene F. Krause. *Taxicab Geometry: An Adventure in Non-Euclidean Geometry*. Dover Publications, New York, 1986.

[10] George B. Dantzig. On the shortest route through a network. *Management Science*, 6(2):187–190, 1960.

[11] Lester R. Ford, Jr. and Delbert R. Fulkerson. *Flows in Networks*. Princeton University Press, Princeton, NJ, 1962.

[12] George L. Nemhauser and Laurence A. Wolsey. *Integer and Combinatorial Optimization*. Wiley, New York, 1988.

[13] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993.

[14] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*, volume 24 of *Algorithms and Combinatorics*. Springer, Berlin, 2003.

[15] Eiji Oki. *Linear Programming and Algorithms for Communication Networks: A Practical Guide to Network Design, Control, and Management*. CRC Press, Boca Raton, FL, 2012.

[16] Leonardo Taccari. Integer programming formulations for the elementary shortest path problem. *European Journal of Operational Research*, 252(1):122–130, 2016.

[17] Saul I. Gass. *Linear Programming: Methods and Applications*. McGraw-Hill, New York, 5th edition, 1985.

[18] Richard M. Karp and James B. Orlin. Parametric shortest path algorithms with an application to cyclic staffing. *Discrete Applied Mathematics*, 3(1):37–45, 1981.

[19] Neal E. Young, Robert E. Tarjan, and James B. Orlin. Faster parametric shortest path and minimum balance algorithms. *Networks*, 21(2):205–221, 1991.

[20] Patricia J. Carstensen. *The complexity of some problems in parametric linear and combinatorial programming*. Ph.D. thesis, University of Michigan, Ann Arbor, MI, 1983.

[21] Richard C. Larson. A hypercube queuing model for facility location and redistricting in urban emergency services. *Computers & Operations Research*, 1(1):67–95, 1974.

[22] Luc Brotcorne, Gilbert Laporte, and Frédéric Semet. Ambulance location and relocation models. *European Journal of Operational Research*, 147(3):451–463, 2003.

[23] Michel Gendreau, Gilbert Laporte, and Frédéric Semet. A dynamic model and parallel tabu search heuristic for real-time ambulance relocation. *Parallel Computing*, 27(12):1641–1653, 2001.

[24] Ariel Orda and Raphael Rom. Shortest-path and minimum-delay algorithms in networks with time-dependent edge-length. *Journal of the ACM*, 37(3):607–625, 1990.

[25] Andrew Makhorin. GLPK—GNU Linear Programming Kit, version 5.0. Free Software Foundation. https://www.gnu.org/software/glpk/.